PCET'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING, PUNE

DEPARTMENT OF INFORMATION TECHNOLOGY

Mini Project Report

On

"Centralized Club Management System"

DATABASE MANAGEMENT SYSTEM LAB

Submitted By

Sanika Patil (124B1F020)

Nishtha Parve (124B1F003)

Guided By: Mrs. Alpana Borse

Academic Year 2025-26

ABSTRACT

The Campus Club Management System is a full-stack web application designed to digitize and streamline the end-to-end operations of student clubs within a college campus. The system addresses the inefficiencies of manual club management – such as scattered event planning, delayed budget approvals, unrecorded attendance, and poor communication – by providing a centralized, role-based platform. The three primary user roles are: club leads (organizers) who can create events, request budgets and venues, mark attendance, and submit post-event reports; faculty coordinators who review and approve budget/venue requests and event proposals; and student members who can register for events, suggest new ideas, and view their attendance history. The backend is built using Node.js, Express.js, and MongoDB, with Mongoose ODM for data modelling. The frontend is developed using React.js and React Router, with a responsive CSS design. Key database concepts implemented include entity-relationship modelling (collections for Users, Clubs, Events, Registrations, BudgetRequests, VenueRequests, EventReports, Attendance, and Notifications), primary and foreign key relationships using MongoDB references, and complex aggregation pipelines for reports such as club activity summaries, budget utilization, and member engagement. The project successfully demonstrates how a well-structured database can power a dynamic web application, reducing manual effort, improving transparency, and fostering better coordination among all campus club stakeholders.

INTRODUCTION

Managing multiple student clubs on a campus involves a wide range of interconnected activities: planning events, securing budgets and venues, tracking attendance, collecting member suggestions, and communicating across different stakeholders. Traditionally, these tasks are carried out using scattered spreadsheets, lengthy email threads, paper forms, or even word of mouth. This fragmented approach leads to several problems: event details are often lost or duplicated, budget approvals are delayed due to lack of visibility, attendance records are incomplete, and coordinators have no real-time overview of club operations. Miscommunication between club leads, faculty coordinators, and student members frequently results in missed deadlines, wasted resources, and reduced participation. Therefore, there is a clear need for a centralized, automated system that brings all club management functions into one platform.

The Campus Club Management System is a full-stack web application that digitises the entire club workflow. It provides role-based access tailored to three types of users:

1. **club leads (organizers)** - create events, request budgets and venues, mark attendance, and submit post-event reports
2. **faculty coordinators** - review and approve these requests
3. **student members** - register for events, suggest new ideas, and view their attendance history.

Every action – from event creation to budget approval to attendance marking – is logged in a central database. This ensures data integrity, eliminates redundancy, and provides a complete audit trail. Real-time notifications keep all users informed about status changes, approvals, and upcoming events.

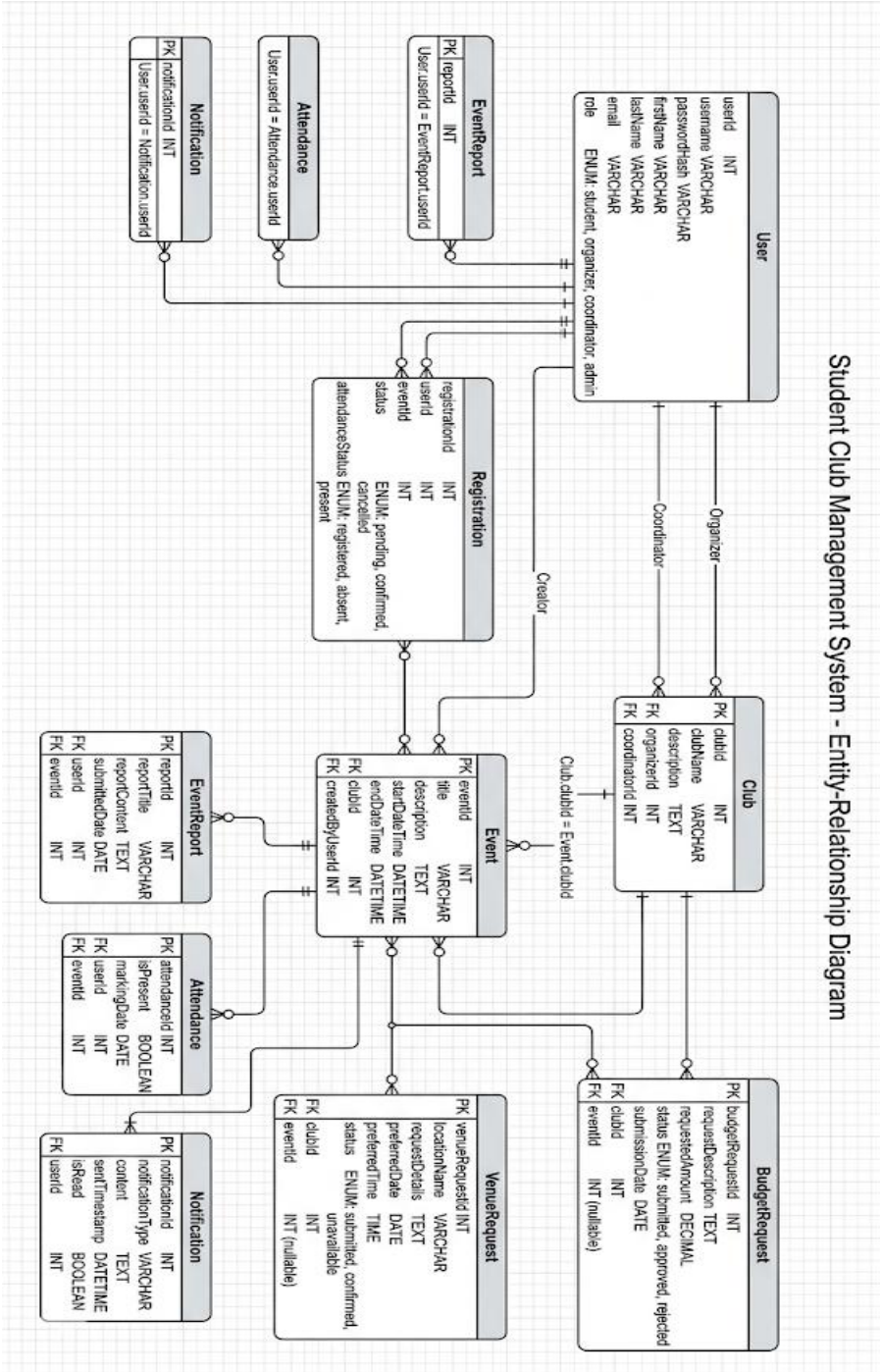
The motivation for this project came directly from the real-world needs of our college's student clubs. Manual coordination often led to important information being lost, event budgets being delayed for weeks, and students missing out on activities due to lack of awareness. By applying core database management concepts such as entity-relationship modelling, normalisation (through properly referenced collections), and transaction management (using MongoDB's atomic operations), we have built a robust, scalable solution. The system not only reduces manual effort but also brings transparency and accountability to campus club management, making it adaptable to any educational institution with minimal changes.

OBJECTIVES

1. **To design a normalized database schema** for a Campus Club Management System that efficiently stores and manages data related to clubs, events, users (students, organizers, coordinators, admins), budget requests, venue requests, attendance, event reports, suggestions, and notifications, ensuring data integrity and reducing redundancy.
2. **To implement CRUD operations** using MongoDB (via Mongoose ODM) for all entities, with proper authentication (JWT) and role-based authorisation (RBAC) to restrict access based on user roles.
3. **To build a full-stack web application** with a React.js frontend (responsive UI, client-side routing) and a Node.js/Express backend, enabling real-time interaction for club leads, coordinators, and students.
4. **To provide role-specific dashboards**:
    a. **Club Lead (Organizer)** – create events, request budgets/venues, mark attendance, submit post-event reports, review member suggestions.
    b. **Coordinator** – approve/reject budget/venue requests, review event proposals, view event reports.
    c. **Student** – register for events, suggest new events, view attendance history, receive notifications.
    d. **Admin** – manage users, clubs, and all event
5. **To implement complex reporting queries** using MongoDB aggregation pipelines, such as:
    a. Club activity summary (events created, registrations, attendance).
    b. Budget utilisation per club.
    c. Member engagement (number of events attended per student).
    d. Event completion report (actual attendees vs registered).
6. **To integrate real-time notifications** for status changes (budget approval, event approval, suggestion response) using a notification collection and polling mechanism.
7. **To ensure a scalable and maintainable architecture** by separating concerns (MVC pattern), using environment variables, and following RESTful API design.

DATABASE DESIGN

1. ER Diagram



2. Tables and Attributes (Relational Schema)

1. **User** (userId, name, email, password, role, clubCode, clubId, isActive)

2. **Club** (clubId, name, description, clubCode, organizerId, coordinatorId, category, createdBy)

3. **Event** (eventId, title, description, date, location, clubId, createdBy, maxParticipants, registrationsCount, status, approvalStatus)

4. **Registration** (regId, userId, eventId, status, attendanceStatus, registeredAt)

5. **BudgetRequest** (reqId, clubId, eventId, requestedBy, amount, purpose, status, coordinatorComment, adminComment)

6. **VenueRequest** (reqId, clubId, eventId, requestedBy, venueName, dateNeeded, durationHours, purpose, status)

7. **EventReport** (reportId, eventId, clubId, submittedBy, actualAttendees, summary, totalExpense, submittedAt)

8. **Attendance** (attId, eventId, clubId, userId, status, markedBy, markedAt)

9. **Notification** (notifId, userId, type, title, message, read, link)

3. Primary Keys and Foreign Keys

Table	Primary Key	Foreign Key(s)
User	userId	clubId → Club(clubId)
Club	clubId	organizerId → User(userId), coordinatorId → User(userId), createdBy → User(userId)
Event	eventId	clubId → Club(clubId), createdBy → User(userId)
Registration	regId	userId → User(userId), eventId → Event(eventId)
BudgetRequest	reqId	clubId → Club(clubId), eventId → Event(eventId), requestedBy → User(userId)
VenueRequest	reqId	clubId → Club(clubId), eventId → Event(eventId), requestedBy → User(userId)
Eve tReport	reportId	eventId → Event(eventId), clubId → Club(clubId), submittedBy → User(userId)
Attendance	attId	eventId → Event(eventId), clubId → Club(clubId), userId → User(userId), markedBy → User(userId)
Notification	notifId	userId → User(userId)

IMPLEMENTATION (MONGODB QUERIES)

1. Create Collections (implicitly done by Mongoose schemas)

In MongoDB, collections are created automatically when the first document is inserted. The schema definitions (models) enforce structure.

```
const userSchema = new mongoose.Schema({
  name: String, email: String, role: String, clubId: { type: ObjectId, ref: 'Club' }
});

const eventSchema = new mongoose.Schema({
  title: String, date: Date, location: String, clubId: { type: ObjectId, ref: 'Club' },
  createdBy: { type: ObjectId, ref: 'User' }, maxParticipants: Number, registrationsCount: Number
});
```

2. Insert sample data

```
db.clubs.insertOne({
  name: "Tech Club", clubCode: "TECH", description: "Technology club",
  organizerId: ObjectId("..."), coordinatorId: ObjectId("..."), createdBy: ObjectId("...")
});

db.events.insertOne({
  title: "Hackathon", date: ISODate("2026-05-20T09:00:00Z"), location: "Seminar Hall",
  clubId: ObjectId("..."), createdBy: ObjectId("..."), maxParticipants: 50, registrationsCount: 0,
  status: "upcoming", approvalStatus: "pending_coordinator"
});
```

3. Query: List all upcoming events with club name (using aggregation lookup)

```
db.events.aggregate([
  { $match: { date: { $gt: new Date() }, status: "upcoming" } },
  { $lookup: { from: "clubs", localField: "clubId", foreignField: "_id", as: "club" } },
  { $unwind: "$club" },
  { $project: { title: 1, date: 1, location: 1, clubName: "$club.name" } },
  { $sort: { date: 1 } }
]);
```

4. Query: Total budget approved per club

```
db.budgetrequests.aggregate([
  { $match: { status: "admin_approved" } },
  { $lookup: { from: "clubs", localField: "clubId", foreignField: "_id", as: "club" } },
  { $unwind: "$club" },
  { $group: { _id: "$club.name", totalApproved: { $sum: "$amount" } } }
]);
```

5. Query: Attendance report for a specific event

```
db.attendances.aggregate([
  { $match: { eventId: ObjectId("201") } },
  { $lookup: { from: "users", localField: "userId", foreignField: "_id", as: "user" } },
  { $unwind: "$user" },
  { $project: { name: "$user.name", status: 1 } }
]);
```

6. Query: Member engagement (number of events attended per member)

```
db.attendances.aggregate([
  { $match: { status: "present" } },
  { $group: { _id: "$userId", eventsAttended: { $sum: 1 } } },
  { $lookup: { from: "users", localField: "_id", foreignField: "_id", as: "user" } },
  { $unwind: "$user" },
  { $project: { name: "$user.name", eventsAttended: 1 } },
  { $sort: { eventsAttended:-1 } }
]);
```

7. Query: Pending budget requests for a specific coordinator (by club)

```
db.budgetrequests.aggregate([
  { $match: { status: "pending" } },
  { $lookup: { from: "clubs", localField: "clubId", foreignField: "_id", as: "club" } },
  { $unwind: "$club" },
  { $match: { "club.coordinatorId": ObjectId("coordinator_user_id") } },
  { $lookup: { from: "users", localField: "requestedBy", foreignField: "_id", as: "requester" } },
  { $unwind: "$requester" },
  { $project: { amount: 1, purpose: 1, requesterName: "$requester.name" } }
]);
```

8. Query: Event completion report – actual vs registered

```
db.events.aggregate([
  { $match: { status: "completed" } },
  { $lookup: { from: "eventreports", localField: "_id", foreignField: "eventId", as: "report" } },
  { $unwind: { path: "$report", preserveNullAndEmptyArrays: true } },
  { $project: {
      title: 1,
```

```
      registeredCount: "$registrationsCount",
      actualAttendees: "$report.actualAttendees",
      attendancePercentage: {
        $cond: [
          { $gt: ["$registrationsCount", 0] },
          { $multiply: [{ $divide: ["$report.actualAttendees", "$registrationsCount"] }, 100] },
          0
        ]
      }
    }
  }
]);
```

9. User Queries (userController.js)

```
// Get logged-in user (with populated club)
User.findById(req.user.id).select("-password").populate("clubId", "name");

// List all users (admin only)
User.find().select("-password").populate("clubId", "name").sort({ createdAt:-1 });

// Update user
User.findByIdAndUpdate(req.params.id, updateData, { new: true });

// Delete user
```

10. Club Queries (clubController.js)

```
// List all clubs (public)
Club.find().populate("organizerId", "name email role").populate("createdBy", "name").sort({ createdAt:-1 });

// Get single club
Club.findById(req.params.id).populate("organizerId", "name email").populate("createdBy", "name");

// Create club
Club.create({ name, description, organizerId, createdBy, category, clubCode });

// Update club
Club.findByIdAndUpdate(req.params.id, updateData, { new: true });

// Delete club
Club.findByIdAndDelete(req.params.id);

// Get club members (users belonging to the club)
User.find({ clubId: req.user.clubId }).select('name email role');

// Get club of logged-in coordinator
Club.findOne({ coordinatorId: req.user.id }).populate('organizerId', 'name email');
```

11. Event Queries (eventController.js)

```
// List all events (public)
Event.find().sort({ date: 1 }).populate("clubId", "name").populate("createdBy", "name role");


// Get single event
Event.findById(req.params.id).populate("clubId", "name description").populate("createdBy",
"name");


// Get my events (organizer)
Event.find({ createdBy: userId }).populate('clubId', 'name').sort({ date:-1 });


// Search event by title (case-insensitive exact match)
Event.findOne({ createdBy: userId, title: { $regex: new RegExp(`^${title}$`, 'i') }
}).populate('clubId', 'name');


// Create event
Event.create({ ...req.body, createdBy: req.user._id, approvalStatus: "pending_coordinator" });


// Update event
Event.findByIdAndUpdate(req.params.id, safeBody, { new: true });


// Delete event
Event.findByIdAndDelete(req.params.id);


// Get pending events for coordinator
Event.find({ clubId: club._id, approvalStatus: "pending_coordinator" }).populate("createdBy",
"name");
```

```
// List event participants (registered users)
Registration.find({ eventId: event._id, status: "registered" }).populate("userId", "name email");
```

12. Registration Queries (registrationController.js)

```
// Register for an event (create or reactivate)
Registration.findOne({ userId: req.user.id, eventId });
// Then either update or create
Registration.create({ userId: req.user.id, eventId, status: "registered" });
// Update existing cancelled registration
existingRegistration.status = "registered"; existingRegistration.save();

// Cancel registration
Registration.findByIdAndUpdate(req.params.id, { status: "cancelled" });

// Get my registrations (student)
Registration.find({ userId: req.user.id, status: "registered" })
  .populate({ path: "eventId", populate: { path: "clubId", select: "name" } })
  .sort({ registeredAt:-1 });
```

13. Budget Request Queries (budgetController.js)

```
// Create budget request
BudgetRequest.create({ clubId, eventId, requestedBy, amount, purpose });

// Get my club's budget requests (organizer)
BudgetRequest.find({ clubId: req.user.clubId }).populate('eventId', 'title').sort('-createdAt');

// Get pending requests for coordinator
```

```
BudgetRequest.find({ clubId: club._id, status: 'pending' }).populate('eventId requestedBy', 'title name');


// Update budget request status

BudgetRequest.findByIdAndUpdate(id, { status, comment, ... }, { new: true });
```

14. Venue Request Queries (venueController.js)

```
// Create venue request

VenueRequest.create({ clubId, eventId, requestedBy, venueName, dateNeeded, durationHours, purpose });


// Get my club's venue requests (organizer)

VenueRequest.find({ clubId: req.user.clubId }).sort('-createdAt');


// Get pending venue requests for coordinator

VenueRequest.find({ clubId: club._id, status: 'pending' }).populate('eventId requestedBy', 'title name');


// Update venue request status

VenueRequest.findByIdAndUpdate(id, { status, coordinatorFeedback });
```

15. Event Report Queries (reportController.js)

```
// Submit event report

EventReport.create({ eventId, clubId, submittedBy, actualAttendees, summary, expenses });


// Get my club's reports (organizer)

EventReport.find({ clubId: req.user.clubId }).populate('eventId', 'title date').sort('-submittedAt');
```

```
// Get all reports for coordinator (by club)

EventReport.find({ clubId: club._id }).populate('eventId submittedBy', 'title name').sort('-submittedAt');
```

16. Suggestion Queries (suggestionController.js)

```
// Create suggestion (student)

EventSuggestion.create({ clubId, suggestedBy, title, description, proposedDate });


// Get club suggestions (organizer)

EventSuggestion.find({ clubId: req.user.clubId }).populate('suggestedBy', 'name email').sort('-createdAt');


// Update suggestion status (organizer)

EventSuggestion.findByIdAndUpdate(id, { status, leadComment });
```

17. Attendance Queries (attendanceController.js)

```
// Mark/update attendance (upsert)

Attendance.findOneAndUpdate(

  { eventId, userId },

  { status, markedBy, markedAt, clubId },

  { upsert: true, new: true }

);


// Get attendance for an event (organizer/coordinator)

Attendance.find({ eventId }).populate('userId', 'name email');


// Get my attendance (student)

Attendance.find({ userId: req.user.id }).populate('eventId', 'title date location').sort('-markedAt');
```

18. Notification Queries (notificationController.js)

```
// Get user notifications
Notification.find({ userId: req.user.id }).sort('-createdAt').limit(50);


// Mark notification as read
Notification.findOneAndUpdate({ _id: id, userId: req.user.id }, { read: true });
```

19. Aggregation Queries for Reports (database_queries.js)

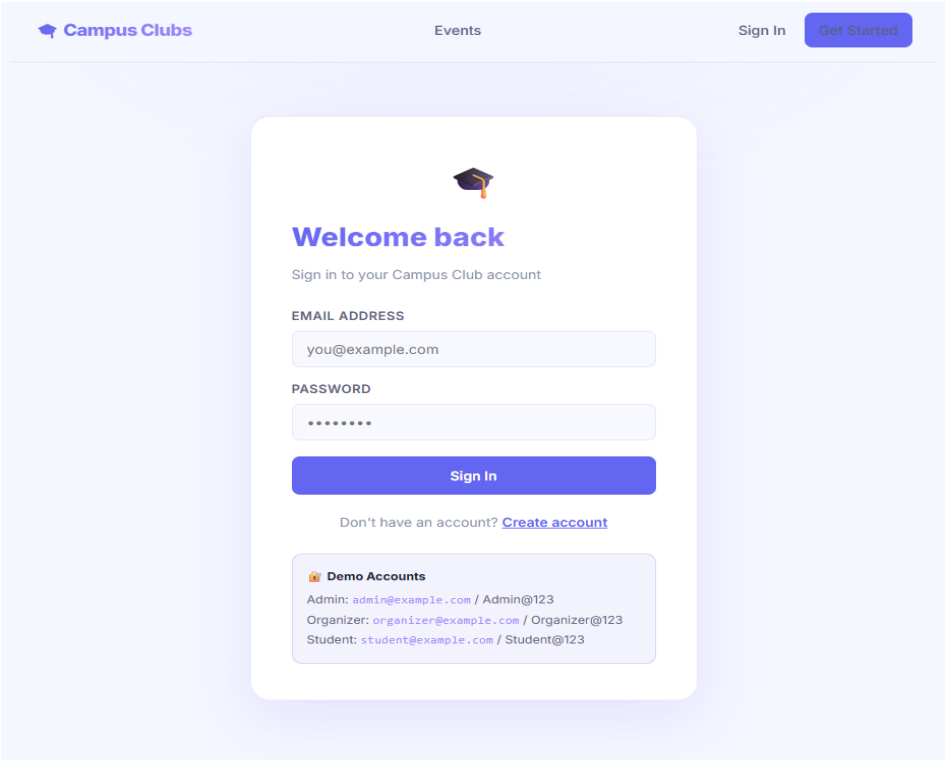
```
// Club activity report
Event.aggregate([
  { $match: { clubId: ObjectId(clubId), date: { $gte: startDate, $lte: endDate } } },
  { $lookup: { from: 'registrations', localField: '_id', foreignField: 'eventId', as: 'regs' } },
  { $addFields: { totalRegistrations: { $size: '$regs' } } },
  { $lookup: { from: 'attendances', localField: '_id', foreignField: 'eventId', as: 'att' } },
  { $addFields: { presentCount: { $size: { $filter: { input: '$att', as: 'a', cond: { $eq: ['$$a.status',
'present'] } } } } } } },
  { $project: { title: 1, date: 1, totalRegistrations: 1, registrationsCount: 1, presentCount: 1 } }
]);


// Budget utilization
BudgetRequest.aggregate([
  { $match: { clubId: ObjectId(clubId), status: 'admin_approved' } },
  { $group: { _id: null, totalApproved: { $sum: '$amount' } } }
]);


// Member engagement
Attendance.aggregate([
```

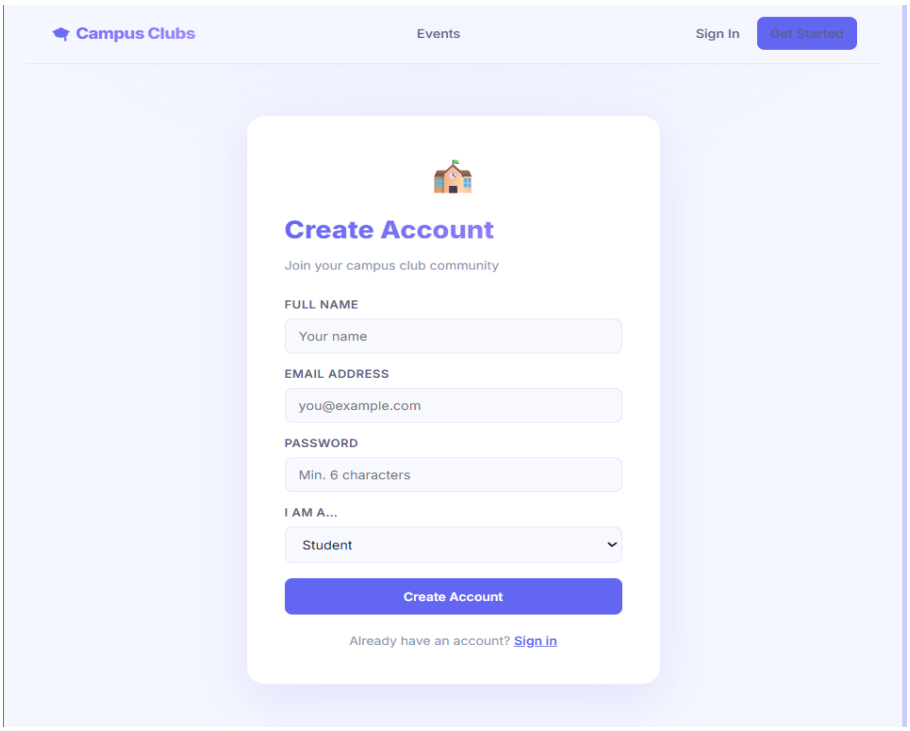
```
  { $match: { clubId: ObjectId(clubId) } },

  { $group: { _id: '$userId', presentCount: { $sum: { $cond: [{ $eq: ['$status', 'present'] }, 1, 0] } } } },

  { $lookup: { from: 'users', localField: '_id', foreignField: '_id', as: 'user' } },

  { $project: { name: { $arrayElemAt: ['$user.name', 0] }, email: { $arrayElemAt: ['$user.email', 0] }, presentCount: 1 } }

]);


// Event expense report

EventReport.aggregate([

  { $match: { clubId: ObjectId(clubId) } },

  { $lookup: { from: 'events', localField: 'eventId', foreignField: '_id', as: 'event' } },

  { $unwind: '$event' },

  { $project: { eventTitle: '$event.title', totalExpense: 1, actualAttendees: 1, submittedAt: 1 } }

]);
```

Results and Output

Screenshot 1: Login & Register Page

There are four types of login options – admin, coordinator, lead, member

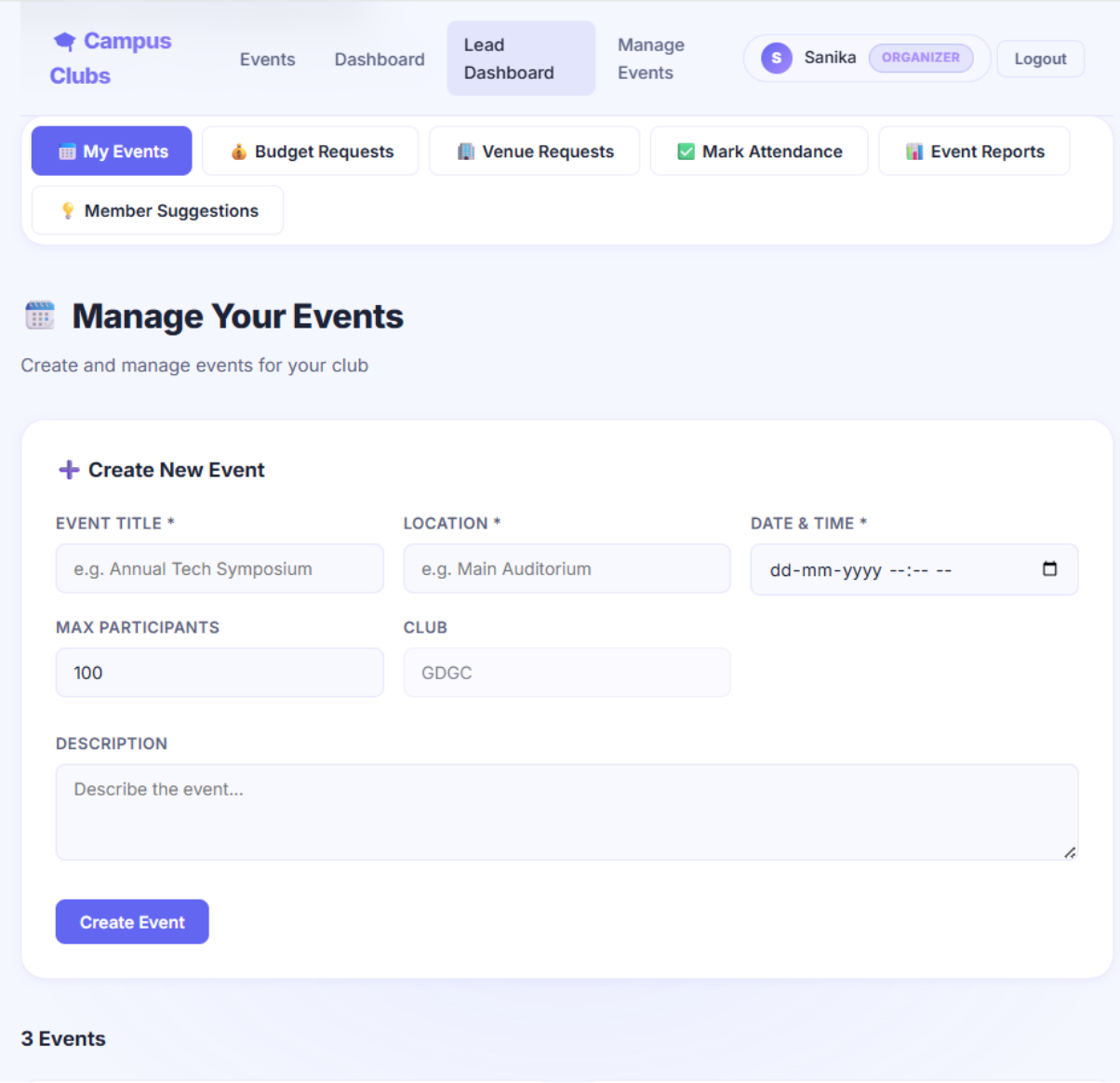




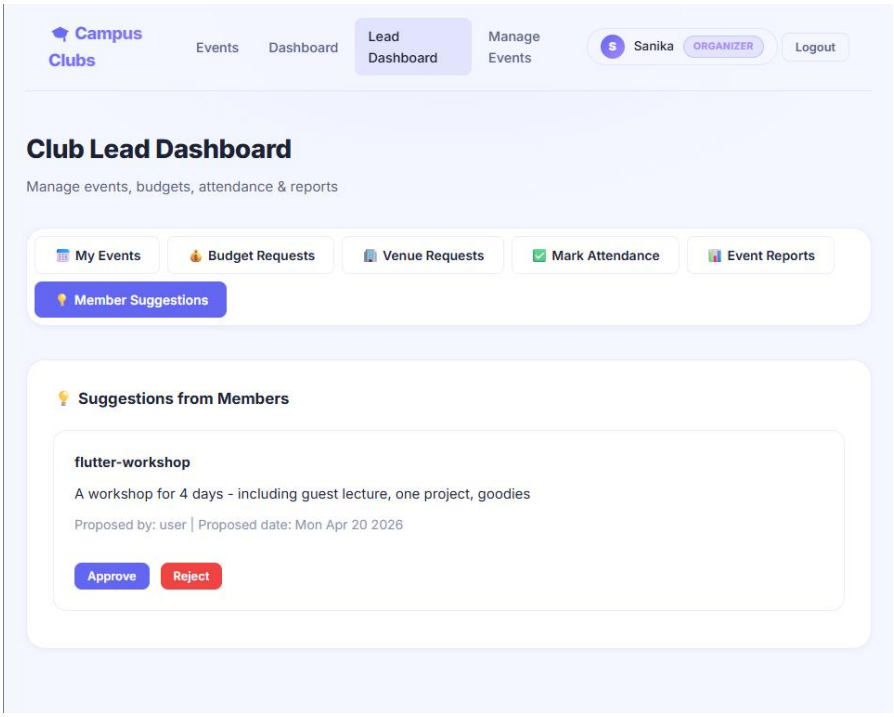
Screenshot 2: Club Lead Dashboard

a) Create Event Form

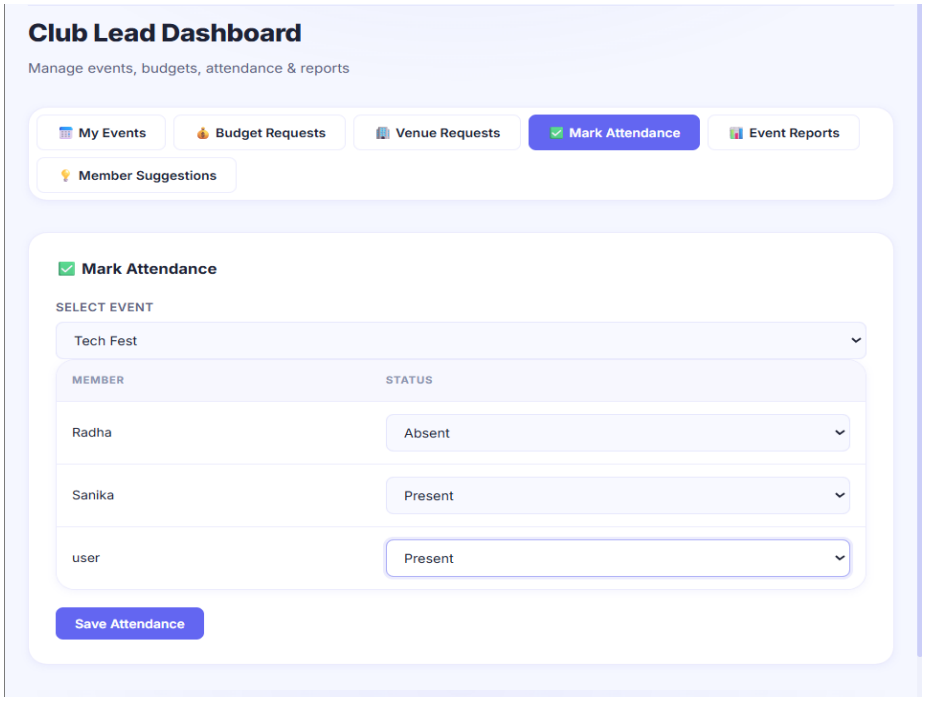
The club lead (organizer) creates a new event by entering the title, date & time, location, maximum participants, and an optional description. Upon submission, the event is saved with status "upcoming" and approval status "pending_coordinator". A notification is sent to the assigned coordinator.



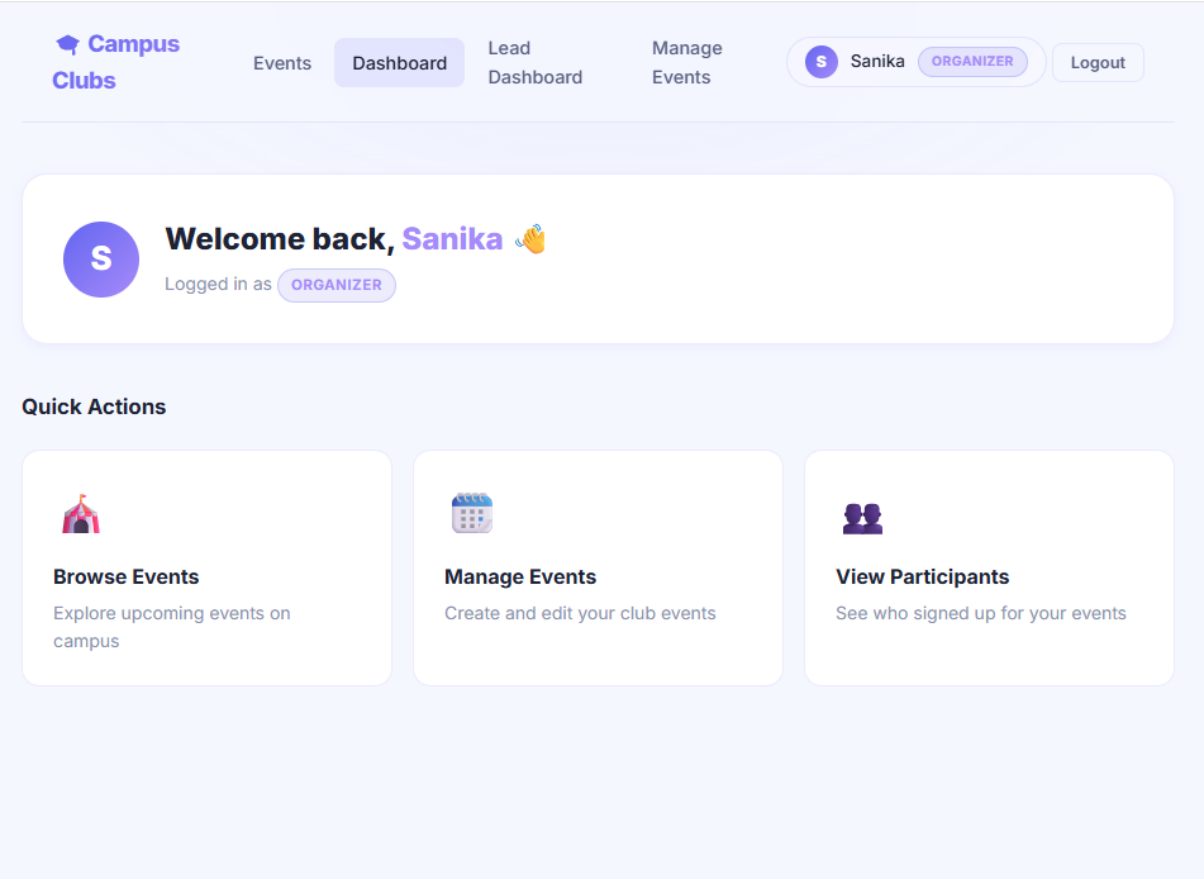
b) Member Event Suggestion notification – Approved & Reject Option



c) Mark Attendance of Member present in event

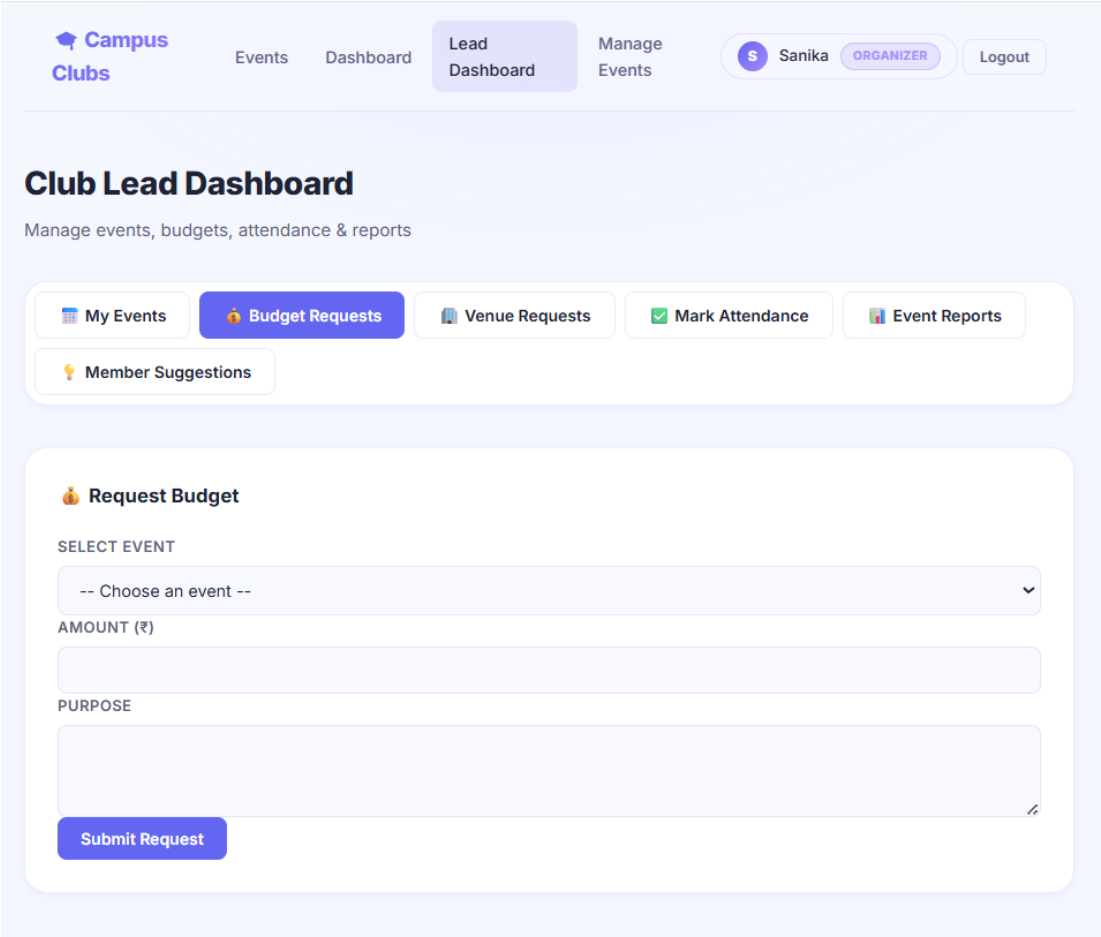


d) Dashboard


Campus Clubs
Events
Dashboard
Lead Dashboard
Manage Events
S
Sanika
ORGANIZER
Logout
S
Welcome back, Sanika
Logged in as
ORGANIZER
Quick Actions
Browse Events
Explore upcoming events on campus
Manage Events
Create and edit your club events
View Participants
See who signed up for your events

Screenshot 2: Budget Request Page with Event Dropdown

The club lead selects an existing event from a dropdown (populated from the /events/my *API), enters the requested amount and purpose, and submits a budget request. The request is stored with status "pending" and a notification is sent to the coordinator.*



Screenshot 3: Coordinator Dashboard – Pending Budget Requests

The coordinator views all pending budget requests for their assigned club, along with event details and requester name. The coordinator can approve or reject each request, optionally adding a comment. Approved requests move to "coordinator_approved" status; rejected requests are marked "rejected".

Campus Clubs | Events | Dashboard | Coordinator Dashboard | R Radha COORDINATOR | Logout

Coordinator Dashboard

Pending approvals for your club

💰 Pending Budget Requests

₹2000 – abcd

Event: TestEvent | Requested by: Sanika

Approve Reject

₹2500 – for the management

Event: Tech Fest | Requested by: Sanika

Approve Reject

Campus Clubs | Events | Dashboard | Coordinator Dashboard | R Radha COORDINATOR | Logout

₹2500 – for the management

Event: Tech Fest | Requested by: Sanika

Approve Reject

🏢 Pending Venue Requests

No pending venue requests.

📊 Submitted Event Reports

hackathon

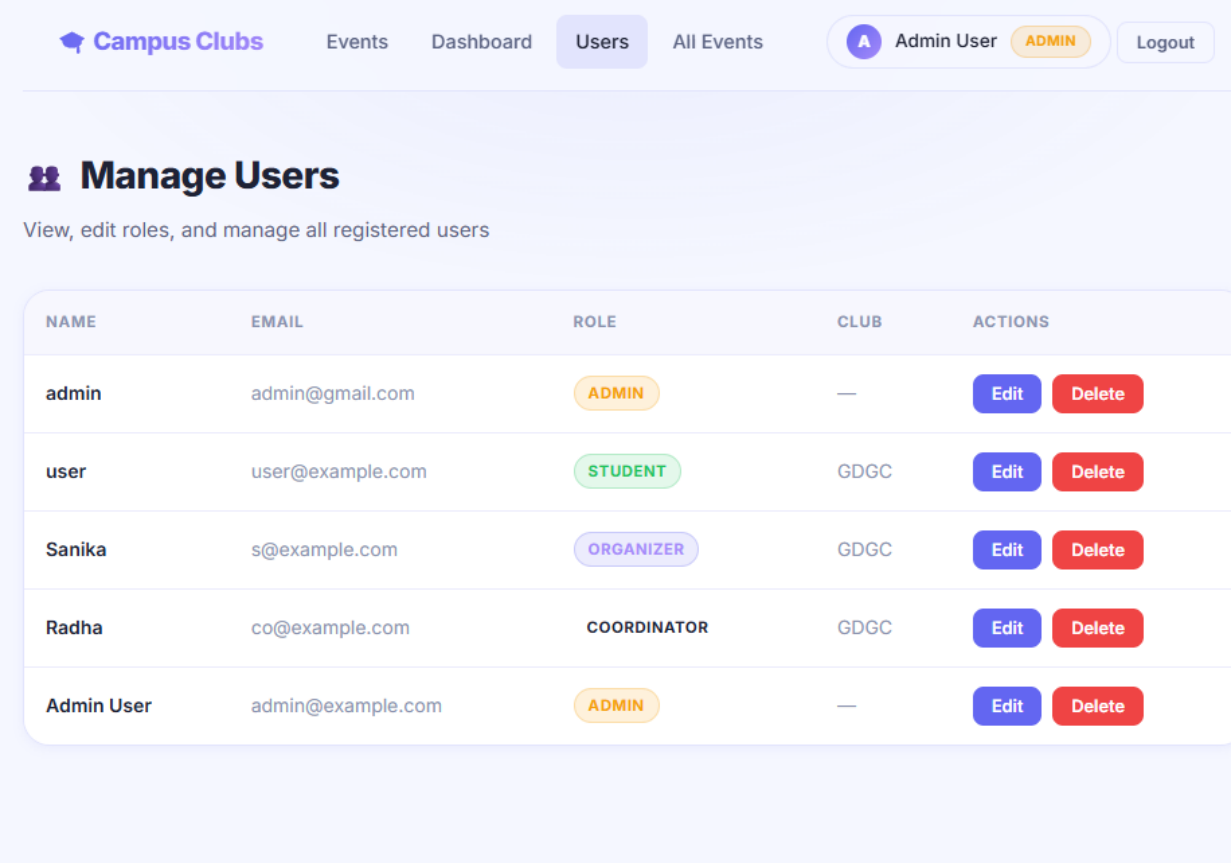
Attendees: 150 | Total Expense: ₹3199

Summary: national level hackathon

Submitted on: 4/19/2026

Screenshot 4: Admin Dashboard

Admin can manage the roles of users such as coordinator, club lead and club member, assign clubs to each user and can see overall details about events and clubs



Screenshot 5: Student Dashboard – Event Suggestion Form

A student member suggests a new event idea by providing a title, description, and optional proposed date. The suggestion is stored with status "pending". The club lead receives a notification and can approve or reject the suggestion from their dashboard

Campus Clubs

Events | Dashboard

user STUDENT | Logout

Discover, register, and participate in events across all clubs

Search by title, location or club...

All Statuses

3 events found

UPCOMING | GDGC

hackathon

hsanx

4/15/2026, 11:02:00 AM

mechanical 5th floor

0 / 100 registered

View Details →

UPCOMING | GDGC

Tech Fest

4/21/2026, 1:14:00 PM

Hall

0 / 47 registered

View Details →

UPCOMING | GDGC

TestEvent

abdc

4/22/2026, 11:58:00 AM

MAin auditorium

0 / 50 registered

View Details →

Campus Clubs

Events | Dashboard | Member Dashboard

user STUDENT | Logout

Suggest an Event

EVENT TITLE

DESCRIPTION / IDEA

PROPOSED DATE (OPTIONAL)

dd-mm-yyyy

Suggest Event

My Upcoming Events

Browse More

You haven't registered for any events yet.

My Attendance

EVENT	STATUS	DATE
Tech Fest	PRESENT	4/19/2026

CONCLUSION

The Campus Club Management System successfully integrates database design with a full-stack web application. It provides role-based dashboards for club leads, coordinators, and students, covering the entire lifecycle of club events – from creation and approval to budget/venue requests, attendance, reporting, and member suggestions. The use of MongoDB allowed flexible schema evolution, while the aggregation framework enabled complex reporting similar to SQL joins. Through this project, we learned how to translate real-world requirements into a robust database schema, implement CRUD operations with proper authentication and authorisation, and build an interactive frontend using React. The system is scalable, secure, and ready for deployment in a campus environment. Future enhancements could include event reminders, certificate generation, and mobile app integration.

References

- MongoDB Documentation: https://docs.mongodb.com
- Mongoose ODM Guide: https://mongoosejs.com
- React Official Docs: https://react.dev
- Express.js Documentation: https://expressjs.com
- Node.js Documentation: https://nodejs.org
- JSON Web Tokens (JWT): https://jwt.io
- Axios Documentation: https://axios-http.com
- Silberschatz, A., Korth, H.F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
- "Student Club Management System: A Web-Based Approach" – International Journal of Computer Applications (IJCA), 2018.
- "NoSQL Databases: A Survey and Comparison" – ACM Computing Surveys, 2019.
- GeeksforGeeks (MongoDB aggregation tutorials)